

APLICACIONES MÓVILES

INTEGRANTES:	Calvache Mateo Camacho Julián Villareal Alexis
FECHA:	14-04-2026
TEMA:	Taller: Arquitectura y Evolución del Desarrollo Móvil

Taller: Arquitectura y Evolución del Desarrollo Móvil

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo Nativo Puro (iOS/Android)

El desarrollo nativo representa el enfoque inicial en la creación de aplicaciones móviles, donde cada plataforma posee su propio conjunto de herramientas, lenguajes y frameworks. En el caso de iOS, se emplean lenguajes como Swift y Objective-C, mientras que en Android se utilizan Java y Kotlin.

Este enfoque permite un acceso directo al hardware del dispositivo, así como a las APIs oficiales del sistema operativo, lo que se traduce en un alto rendimiento y una experiencia de usuario optimizada. Sin embargo, implica un mayor costo de desarrollo y mantenimiento, ya que se deben crear aplicaciones separadas para cada plataforma.

En la actualidad, el desarrollo nativo sigue siendo altamente relevante, especialmente en aplicaciones que requieren alto rendimiento, seguridad o uso intensivo de recursos, como aplicaciones bancarias, videojuegos o soluciones de realidad aumentada.

2. La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic inicial)

Con el objetivo de reducir costos y unificar el desarrollo, surgieron soluciones híbridas basadas en tecnologías web, como Apache Cordova, PhoneGap y las primeras versiones de Ionic.

Estas herramientas permiten desarrollar aplicaciones utilizando HTML, CSS y JavaScript, que se ejecutan dentro de un contenedor WebView, es decir, un navegador embebido en la aplicación.

Aunque este enfoque facilitó la adopción del desarrollo móvil por parte de desarrolladores web, presentó limitaciones importantes en términos de rendimiento, fluidez y experiencia de usuario. En la actualidad, su relevancia ha disminuido considerablemente, quedando principalmente para aplicaciones simples o prototipos.

3. El Puente Nativo-JavaScript y otros puentes

APLICACIONES MÓVILES

Posteriormente, surgieron frameworks como React Native, que introdujeron un enfoque intermedio entre lo nativo y lo híbrido.

En este modelo, la lógica de la aplicación se desarrolla en JavaScript, mientras que la interfaz se renderiza utilizando componentes nativos. La comunicación entre ambos entornos se realiza mediante un “puente” (bridge), que permite la interacción entre el código JavaScript y las APIs nativas.

Este enfoque ofrece un mejor rendimiento que las soluciones basadas en WebView y permite reutilizar código entre plataformas. No obstante, el uso del puente puede generar latencias en aplicaciones complejas o con alta frecuencia de actualización.

Actualmente, este paradigma sigue siendo relevante, aunque está evolucionando hacia arquitecturas más eficientes que reducen o eliminan la dependencia del bridge.

4. El Motor de Renderizado Propio

Una evolución significativa en el desarrollo multiplataforma es representada por herramientas como Flutter.

A diferencia de los enfoques anteriores, Flutter no utiliza componentes nativos del sistema operativo, sino que implementa su propio motor de renderizado gráfico (basado en Skia), lo que le permite dibujar toda la interfaz de usuario de forma independiente. Este framework utiliza el lenguaje Dart.

Entre sus principales ventajas se encuentran la consistencia visual entre plataformas y un alto rendimiento, al eliminar la necesidad de un puente de comunicación. Sin embargo, puede presentar desventajas como un mayor tamaño de las aplicaciones y menor integración “natural” con el sistema operativo.

En la actualidad, Flutter es una de las soluciones multiplataforma más relevantes y en crecimiento.

5. El Futuro: Foldables, AR y Wearables

Las nuevas tecnologías están transformando el desarrollo móvil al romper el paradigma tradicional de una sola pantalla estática.

Foldables (Pantallas plegables)

Los dispositivos con pantallas plegables requieren interfaces altamente adaptativas, capaces de responder en tiempo real a cambios en el tamaño y la forma de la pantalla. Esto implica el uso de arquitecturas reactivas que mantengan el estado de la aplicación sin pérdida de información durante las transiciones.

En este contexto, el desarrollo nativo ofrece el mejor soporte, aunque frameworks como Flutter también han demostrado una buena capacidad de adaptación. Por su parte, soluciones como React Native dependen en mayor medida de implementaciones específicas.

APLICACIONES MÓVILES

AR / XR (Realidad Aumentada y Extendida)

La realidad aumentada introduce requisitos técnicos más exigentes, como el renderizado 3D en tiempo real, el uso intensivo de la GPU y cálculos matemáticos complejos relacionados con el posicionamiento y la percepción del entorno.

Entre las principales tecnologías utilizadas se encuentran ARKit, ARCore y motores gráficos como Unity.

Las herramientas multiplataforma tradicionales, como React Native o Flutter, no están diseñadas para este tipo de procesamiento, por lo que suelen depender de integraciones con motores externos o código nativo. Por esta razón, el desarrollo de aplicaciones de realidad aumentada continúa siendo predominantemente nativo o basado en motores especializados.

Wearables (Dispositivos portables)

Los dispositivos wearables, como relojes inteligentes o gafas, requieren aplicaciones ligeras, eficientes en consumo energético y con interfaces simplificadas. Para su desarrollo, se utilizan SDKs específicos como los de watchOS o Wear OS.

6. Conclusión General

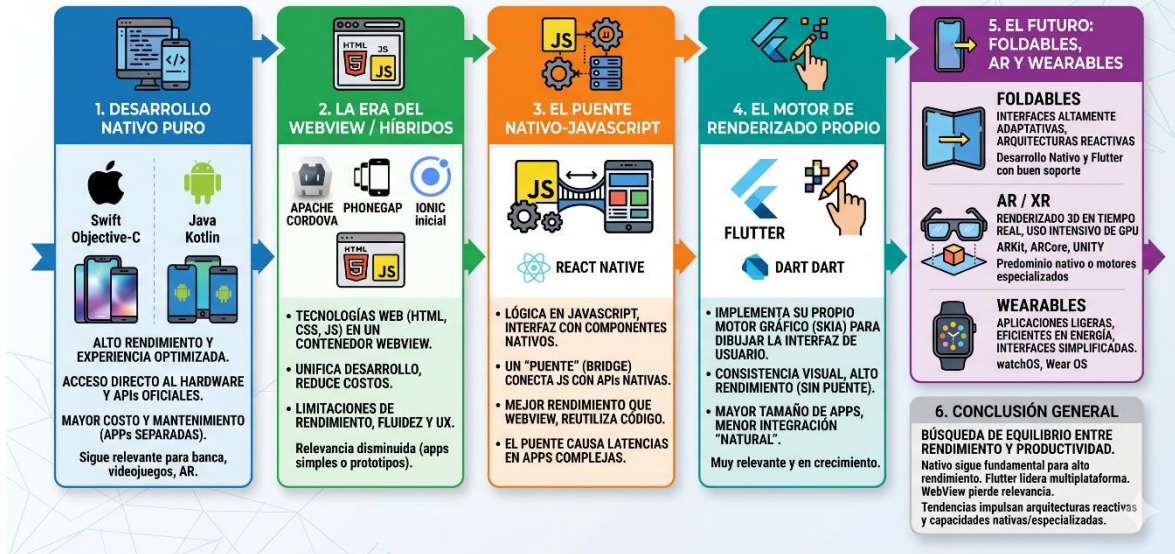
La evolución del desarrollo móvil refleja una constante búsqueda de equilibrio entre rendimiento y productividad. Mientras que el desarrollo nativo ofrece el máximo control y eficiencia, las soluciones multiplataforma han permitido reducir costos y acelerar los tiempos de desarrollo.

En la actualidad, el desarrollo nativo continúa siendo fundamental para aplicaciones de alto rendimiento, mientras que herramientas como Flutter lideran el desarrollo multiplataforma. Por otro lado, las soluciones basadas en WebView han perdido relevancia.

Finalmente, las nuevas tendencias tecnológicas, como los dispositivos plegables y la realidad aumentada, están impulsando nuevamente el uso de arquitecturas más reactivas y una mayor dependencia de capacidades nativas y motores especializados.

APLICACIONES MÓVILES

LA EVOLUCIÓN DEL DESARROLLO MÓVIL: UN VIAJE ARQUITECTÓNICO



Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

Caso de Estudio A: Roblox

1. Ingeniería Inversa de la decisión:

- ¿Por qué Roblox utiliza C++ como motor y Luau en vez de JavaScript o Dart?

El corazón de la aplicación móvil (renderizado, físicas, red, seguridad) está escrito en C++ puro ya que les permite compilar casi directamente a nivel de hardware interactuando con las APIs gráficas de iOS (Metal) y Android (Vulkan/OpenGL ES). Además, Roblox no usa JavaScript ni Dart. Crearon Luau, un lenguaje derivado de Lua, pero estrictamente tipado y optimizado para ser absurdamente rápido y seguro (sandboxed). virtual Luau embebida en el cliente móvil

2. Análisis de Rendimiento y Arquitectura:

- ¿Qué desafíos arquitectónicos enfrenta la app de Roblox al manejar animaciones complejas usando el motor de C++?

El mayor desafío de Roblox móvil no es mostrar una lista de chats, sino descargar dinámicamente entornos 3D, compilar código en tiempo real y mantener 60 FPS en un teléfono de gama baja. Por ello, al usar react-lua y Rodux (gestor de estado unidireccional), aplican principios de Clean Architecture al ecosistema del videojuego. La UI móvil se

APLICACIONES MÓVILES

programa de forma declarativa. Cuando un estado cambia (ej. monedas del usuario), el Virtual DOM de react-lua calcula la diferencia (diffing) y solo actualiza los componentes estrictamente necesarios durante el ciclo de RenderStep del motor, evitando caídas de frames.

- **¿Qué cuello de botella tuvo que resolver Roblox?**

Roblox tuvo que resolver el cuello de botella que se da si el código Luau hace demasiadas llamadas al API de C++ en un solo frame. Para mitigar esto, su arquitectura multihilo utiliza un Task Scheduler riguroso que divide la carga entre el HeartBeat (lógica y físicas en hilos secundarios) y el RenderStep (dibujado final en el hilo principal).

3. EL Desafío de las Nuevas Pantallas

- **¿Roblox tendría más facilidad de adaptación según su arquitectura subyacente en el caso de querer adaptar su aplicativo móvil a Realidad Aumentada Interactiva?**

Si el mercado migra masivamente a visores de Realidad Mixta/Aumentada interactiva, la arquitectura de Roblox tiene una ventaja abrumadora e indiscutible sobre cualquier framework móvil estándar (React Native o Flutter) dado que la UI de Roblox está construida sobre primitivas de motor 3D. Al estar fundamentado en un motor de físicas y renderizado espacial nativo (C++), portar Roblox a un entorno AR (como ya lo hicieron con Meta Quest) implica simplemente reconfigurar el renderizador estereoscópico.

Caso de Estudio B: X (anteriormente llamado Twitter)

1. Ingeniería Inversa de la Decisión

- **¿Por qué X utiliza un desarrollo Nativo (Swift/Kotlin) con un núcleo en C++ en lugar de frameworks como React Native o Flutter?**

Por el rendimiento crítico del feed o Timeline. Al ser una lista infinita de elementos complejos (video, hilos, encuestas), X requiere un control granular del reciclaje de celdas y la gestión de memoria que solo lo nativo ofrece para alcanzar los 120Hz de fluidez. El uso de C++ permite que la lógica pesada de datos sea compartida y ultra veloz, mientras que la UI nativa garantiza una respuesta táctil inmediata sin capas de abstracción.

2. Análisis de Rendimiento y Arquitectura

- **¿Qué cuello de botella en el Hilo Principal (Main Thread) resolvió X para lograr un scroll fluido de contenido complejo en diversas gamas de teléfonos?**

APLICACIONES MÓVILES

La saturación causada por la decodificación de medios y el cálculo de diseños o layouts simultáneos durante el desplazamiento. X implementó un sistema de Layout Pre-calculado en hilos secundarios. Así, cuando el usuario llega al mensaje, el hilo principal ya conoce las dimensiones exactas y solo debe dibujar el elemento, evitando caídas de frames o FPS.

3. El Desafío de las Nuevas Pantallas

- **Ante la llegada de lentes de Realidad Aumentada, ¿qué ventaja ofrece la arquitectura de X para adaptarse a este hardware?**

Ventaja competitiva por acceso directo al hardware. Al ser nativa, la aplicación utiliza directamente las APIs de bajo nivel como ARKit o ARCore y el motor gráfico del sistema sin depender de plugins de terceros. Esto permite integrar la interfaz de hilos en un espacio 3D con una latencia casi nula, factor crítico en realidad aumentada para evitar mareos en el usuario.

Veredicto Crítico

- **Como futuros ingenieros de software: Si estuvieran creando hoy una aplicación que requiere interactuar con mapas 3D interactivos, notificaciones de alta frecuencia y debe soportar pantallas plegables... ¿Elegirían Nativo, React Native o Flutter?**

Elegiríamos NATIVO, debido a:

- Mapas 3D: El uso intensivo de la GPU mediante Metal o Vulkan es más eficiente y estable sin las capas de abstracción de frameworks multiplataforma.
- Notificaciones: La gestión precisa de Background Services nativos asegura la recepción de datos constantes sin que el sistema operativo cierre la aplicación para ahorrar batería.
- Plegables: Las APIs nativas como Jetpack WindowManager ofrecen soporte inmediato para bisagras y cambios de tamaño dinámicos mucho antes de que lleguen parches estables a otras herramientas.

Finalmente respondan esta pregunta al final:

¿Qué es un aplicativo nativo, hay varios, como se clasifican?

Es un software desarrollado exclusivamente para una plataforma o sistema operativo (SO) específico, utilizando únicamente los lenguajes de programación, las bibliotecas (SDKs) y los entornos de desarrollo integrados (IDEs) oficiales y soportados directamente por el fabricante de esa plataforma.

Clasificación:

APLICACIONES MÓVILES

- Nativos Móviles (Mobile Native)
- Nativos de Escritorio (Desktop Native)
- Nativos de Sistemas Embebidos y Wearables
- WebAssembly (WASM)